

ClinicFlow Desktop + Offline First + Local Server Roadmap

Project Goal

Transform the existing ClinicFlow MERN SaaS into a desktop-based offline-first clinic management system that supports:

- Cloud-only operation
 - Local-server-only operation
 - Hybrid operation (Local + Cloud)
 - Multi-receptionist appointment booking
 - Automatic synchronization
 - Internet outage tolerance
-

Final Architecture

Desktop Client (Electron + React)

↓

Data Service Layer

↓

Local Server (Node.js + SQLite)

↓

Sync Engine

↓

Cloud API (Node.js + MongoDB Atlas)

Scenario Handling

Scenario 1: Cloud Available, Local Server Unavailable

Desktop App ↓ Cloud API ↓ MongoDB Atlas

Behavior:

- CRUD operations execute directly on cloud.
 - Operations are logged.
 - When local server becomes available, cloud data syncs to local database.
-

Scenario 2: Local Server Available, Cloud Unavailable

Desktop App ↓ Local Server ↓ SQLite

Behavior:

- CRUD operations continue normally.
 - All changes are added to sync queue.
 - Once cloud returns, queued operations are synchronized.
-

Scenario 3: Both Available

Desktop App ↓ Local Server ↓ SQLite ↓ Sync Engine ↓ Cloud

Behavior:

- Local server becomes primary write source.
 - Faster response time.
 - Offline capability maintained.
 - Background cloud synchronization.
-

Phase 1: Electron Desktop Application

Objective

Convert existing React frontend into desktop application.

Tasks

- Install Electron
- Create Electron Main Process
- Create Preload Process
- Package React inside Electron
- Generate Windows installer

Deliverables

- ClinicFlow Desktop.exe
 - Login functionality
 - Existing cloud functionality working
-

Phase 2: Local Server Development

Objective

Create dedicated local server application.

Tech Stack

- Node.js
- Express
- SQLite

Tasks

- Create Local Server project
- Setup SQLite database
- Create REST APIs
- Implement LAN accessibility
- Create server dashboard

Deliverables

- ClinicFlow Server.exe
 - SQLite database creation
 - API endpoints
-

Phase 3: Shared Database Architecture

Objective

Allow multiple receptionists to access same data.

Tasks

- Central SQLite database
- LAN communication
- Multi-user access testing

Deliverables

- Appointment sharing
 - Patient sharing
 - Billing sharing
-

Phase 4: Connection Manager

Objective

Automatically detect available data source.

Priority Order

1. Local Server
2. Cloud API

Tasks

- Local server health check
- Cloud health check
- Automatic switching

Deliverables

- Seamless connection switching
-

Phase 5: Universal ID System

Objective

Prevent synchronization conflicts.

Tasks

Replace:

- Auto Increment IDs

With:

- UUID v4

Example:

f47ac10b-58cc-4372-a567-0e02b2c3d479

Deliverables

- Unique IDs across cloud and local systems
-

Phase 6: Sync Queue Implementation

Objective

Track unsynchronized changes.

SQLite Table

sync_queue

Fields:

- id
- entity
- action
- payload
- status
- created_at

Actions:

- CREATE
- UPDATE
- DELETE

Deliverables

- Reliable synchronization tracking
-

Phase 7: Cloud → Local Synchronization

Objective

Keep local server updated.

Tasks

- Detect local server startup
- Fetch cloud records
- Insert missing records
- Update modified records

Deliverables

- Local server catches up automatically
-

Phase 8: Local → Cloud Synchronization

Objective

Push offline changes to cloud.

Tasks

- Background sync worker
- Retry mechanism
- Batch processing
- Conflict detection

Deliverables

- Automatic cloud synchronization
-

Phase 9: Conflict Resolution

Objective

Prevent data corruption.

Record Structure

{ id, version, updatedAt }

Rules:

- Higher version wins
- Latest timestamp wins if versions equal

Deliverables

- Reliable data consistency
-

Phase 10: Local Network Discovery

Objective

Automatically locate local server.

Technology

- mDNS
- Bonjour

Example

clinicflow.local

Deliverables

- No manual IP configuration

Phase 11: Backup System

Objective

Protect clinic data.

Tasks

- Daily backup
- Backup rotation
- Restore functionality

Deliverables

Backups folder:

- backup-1.db
- backup-2.db
- backup-3.db

Phase 12: Security

Tasks

- JWT Authentication
- Password Hashing
- Helmet
- Rate Limiting
- Audit Logs

Audit Events:

- Appointment Created
- Appointment Updated
- Appointment Deleted
- Patient Updated

Deliverables

- Secure local deployment
-

Phase 13: Packaging

Server Package

ClinicFlowServerSetup.exe

Includes:

- Node Runtime
- SQLite
- Database Initialization

Desktop Package

ClinicFlowDesktopSetup.exe

Includes:

- Electron App
 - React Frontend
-

Phase 14: Demo Milestone

Demo Flow

1. Install Server.exe
2. Start Local Server
3. Verify SQLite created
4. Install Desktop App
5. Connect Desktop App
6. Create Appointment
7. Open Second PC
8. View Same Appointment
9. Disconnect Internet
10. Continue CRUD Operations
11. Reconnect Internet
12. Automatic Synchronization

Expected Outcome

- Multi-user support
- Offline-first functionality
- Cloud synchronization
- Desktop application experience

- Production-ready architecture foundation